ChatGPT said:

Microservices Patterns: DSA, LLD, and HLD Perspectives

Microservices patterns are reusable architectural solutions to problems that arise when a system is decomposed into independently deployable services communicating over a network.

For senior, Staff-level, and Architect interviews, the important skill is not merely memorizing pattern definitions. You need to understand:

- **DSA (Data Structures and Algorithms):** What algorithms, data structures, complexity characteristics, and concurrency controls make the pattern work?
- **LLD (Low-Level Design):** How do you implement the pattern using classes, interfaces, design patterns, transactions, and error handling?
- **HLD (High-Level Design):** Where does the pattern fit into the architecture? How does it affect scalability, availability, consistency, latency, and operational complexity?

A useful way to think about the three levels is:

DSA — Mechanism

Queues, hash maps, state machines, graph traversal, backoff algorithms, concurrency, complexity analysis.

LLD — Implementation

Interfaces, components, dependency injection, transaction boundaries, persistence, state transitions, testability.

HLD — Architecture

Service boundaries, network communication, data ownership, distributed consistency, failure isolation, capacity planning.

We will use an e-commerce platform as a running example: `Order`, `Payment`, `Inventory`, `Shipping`, and `Notification` services.

1. Microservices pattern catalog

These are the major patterns you should understand for senior backend and architecture interviews.

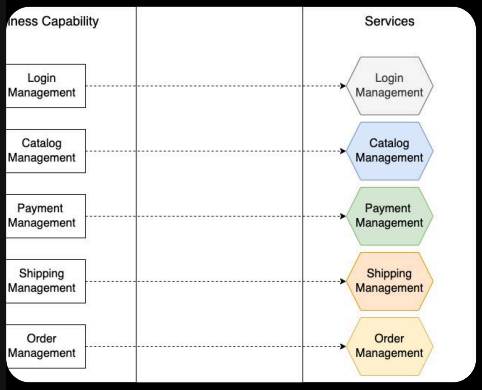
Category	Important patterns
Service decomposition	Decompose by business capability, DDD bounded contexts, Strangler Fig, modular monolith

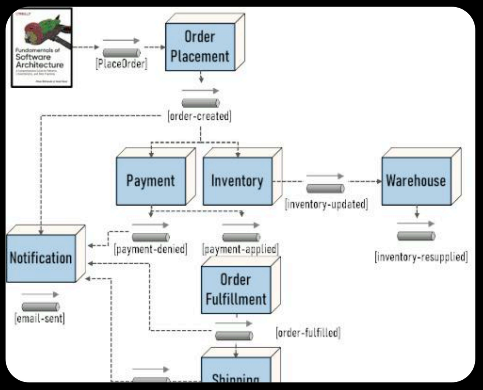
Communication	API Gateway, BFF, synchronous request-response, asynchronous messaging, request-reply
Resilience	Timeout, Retry, Circuit Breaker, Bulkhead, Rate Limiting, Fallback
Data management	Database per Service, Saga, Transactional Outbox, CQRS, Event Sourcing
Consistency and messaging	Idempotent Consumer, Deduplication, Inbox, Ordering, DLQ
Discovery and routing	Service Discovery, Client-side/Server-side Load Balancing
Performance	Caching, Read Replicas, Materialized Views, Aggregation
Security	Service-to-Service Authentication, OAuth 2.0, mTLS, Zero Trust
Observability	Distributed Tracing, Correlation IDs, Metrics, Structured Logging
Deployment	Sidecar, Ambassador, Canary, Blue-Green, Health Checks

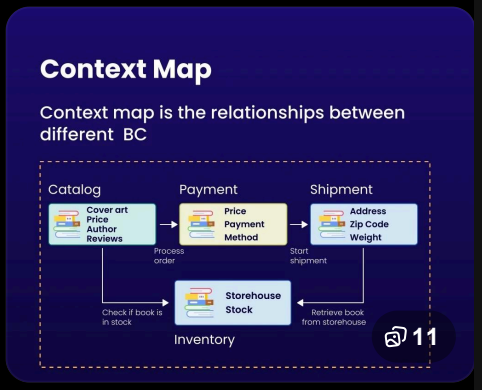
The key is to connect these patterns. For example, a Saga commonly works with asynchronous messaging, an Outbox, idempotent consumers, retries, and observability.

2. Service decomposition patterns

2.1 Decompose by business capability and bounded context







Problem: A large application has tightly coupled modules, shared database tables, and coordinated releases.

Solution: Divide the application into services around business capabilities and domain boundaries, rather than simply splitting by technical layers.

For example:

- Order Service owns order lifecycle and order status.
- Payment Service owns payment attempts, authorizations, and refunds.
- Inventory Service owns stock reservations and availability.

- Shipping Service owns shipment creation and tracking.

DSA perspective

This is not primarily an algorithmic problem, but graph theory helps reason about coupling.

Model modules as a directed graph:

$$G = (V, E)$$

- V: business modules or services.
- E: dependencies between them.

A decomposition should reduce problematic cross-boundary dependencies while keeping cohesive business logic together.

Useful concepts include:

- Dependency graphs and strongly connected components.
- Graph partitioning and minimizing cross-partition communication.
- Dependency direction and cycle detection.

A simple graph-partitioning objective might be:

$$\min \left(\sum_{\text{cross-boundary edges}} w_e + \lambda \sum_{\text{within-boundary pairs}} d_{ij} \right)$$

Here, the first term represents cross-boundary communication cost, while the second represents the penalty for separating tightly related domain concepts. Real service decomposition is more nuanced than this simplified objective.

LLD perspective

Use domain entities, aggregates, repositories, and application services to enforce ownership.

Java

```java
public interface OrderRepository {
    Order findById(OrderId id);
    void save(Order order);
}

public interface PaymentGateway {
```

```
    PaymentResult authorize(Money amount, PaymentId id);
}
```

The Order domain should not directly manipulate Payment Service's database entities. Cross-service communication should happen through an explicit contract.

HLD perspective

Choose boundaries based on:

- Business capability and data ownership.
- Independent deployment and scaling needs.
- Team ownership.
- Change frequency.
- Availability and consistency requirements.

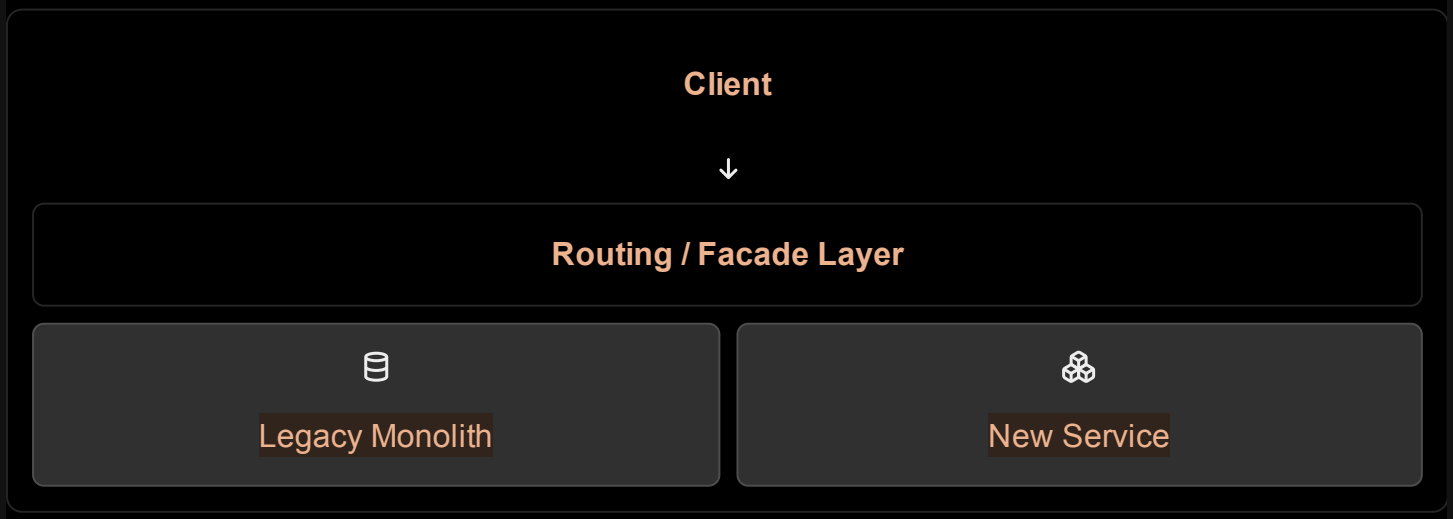
Trade-off: Too few services cause excessive coupling. Too many create a distributed monolith, where almost every operation requires multiple network calls.

Interview takeaway: Start with a modular monolith when boundaries are unclear. Extract microservices when independent deployment, scaling, or ownership justifies the distributed-system cost.

2.2 Strangler Fig pattern

Problem: Replacing a legacy monolith in one large migration is risky.

Solution: Route selected functionality to new services incrementally while keeping the old application running.



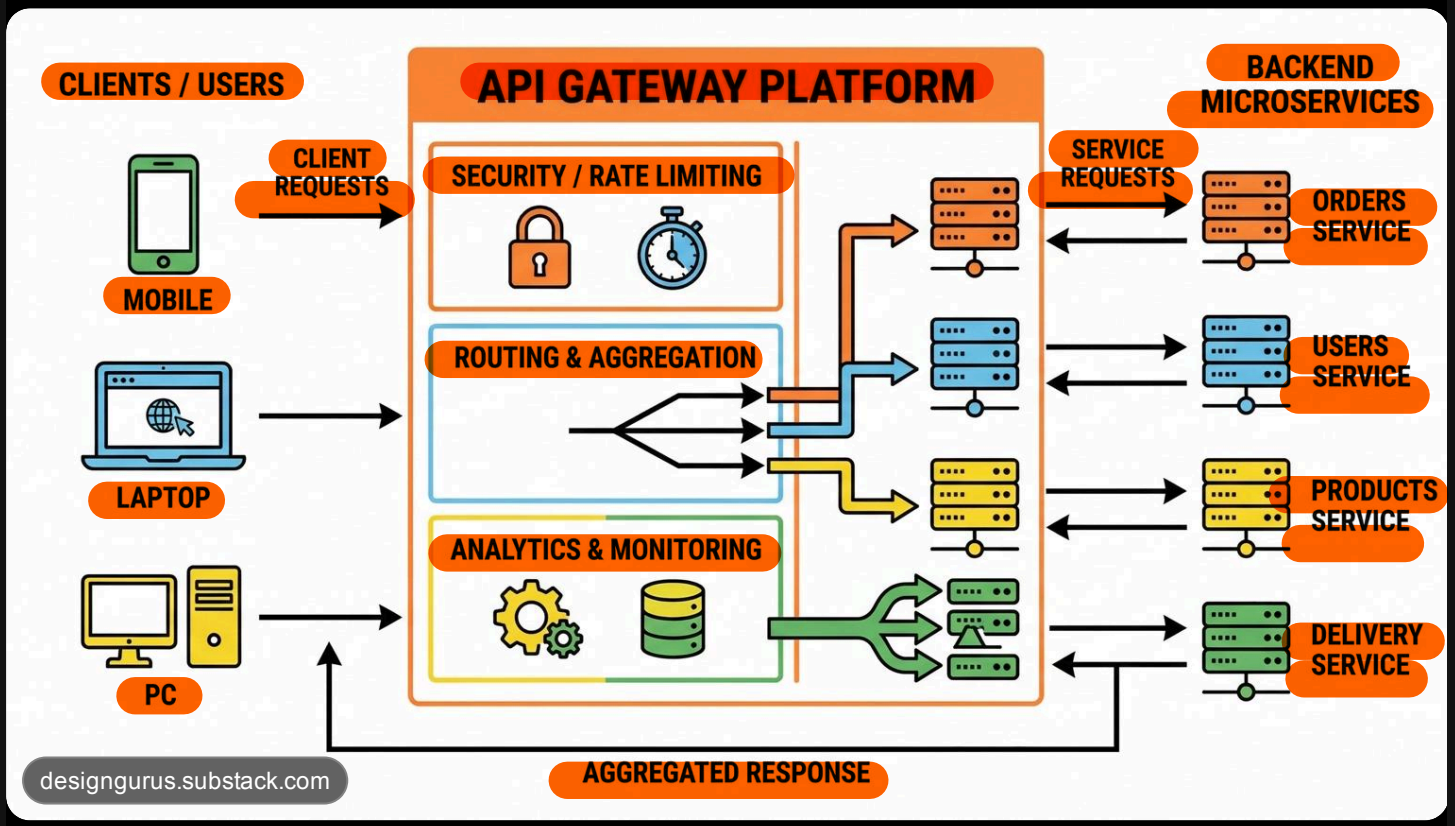
- **DSA:** Routing rules, lookup maps, feature flags, and dependency graphs.
- **LLD:** Facade interfaces, routing strategies, adapters, and contract tests.
- **HLD:** Traffic migration, data ownership transition, rollback, and coexistence.

The hardest part is often data migration and preventing the old and new implementations from becoming competing sources of truth.

---

3. Communication patterns

3.1 API Gateway



Problem: Clients must otherwise know multiple service endpoints and implement authentication, routing, throttling, and other cross-cutting concerns.

Solution: Introduce a gateway as the entry point for client traffic.

Example:

Plain text

```
Mobile App ─┐
Web App ────┼─> API Gateway ──> Order Service
Partner API ┘                 ├─> Payment Service
                              └─> Product Service
```

Perspective	What to understand

DSA	Routing-table lookup, hash maps, token-bucket rate limiting, connection management
LLD	Route definitions, filters, authentication middleware, policy interfaces, exception translation
HLD	Public entry point, TLS termination, routing, load balancing, gateway redundancy and scaling

A simple routing lookup using a hash map is typically $O(1)$ average time for exact-key lookup. Prefix routing, route precedence, and complex matching can have different costs.

Trade-offs: A gateway simplifies clients but adds a network hop and can become a bottleneck if it is not scaled or designed properly.

Senior-level insight: Do not place every business rule inside the gateway. Authentication enforcement, routing, and coarse-grained policies belong there; order and payment business invariants belong in their respective services.

3.2 Synchronous request-response

Problem: A caller needs an immediate result from another service.

Solution: Use HTTP/REST or gRPC.

```
Plain text
Client → Order Service → Payment Service → Response
```

- **DSA:** Latency composition, bounded concurrency, connection pools, queueing, timeout budgets.
- **LLD:** HTTP clients, typed DTOs, error mapping, deadlines, request-scoped correlation IDs.
- **HLD:** Service dependencies, critical-path latency, availability propagation, protocol selection.

For a sequential request path:

$$T_{\text{total}} \approx T_{\text{order}} + T_{\text{payment}} + T_{\text{network}}$$

This approximation assumes the calls are sequential and excludes additional queueing and retry delays.

If each of four independent dependencies has 99.9% availability and every dependency is required for success, the combined availability is approximately:

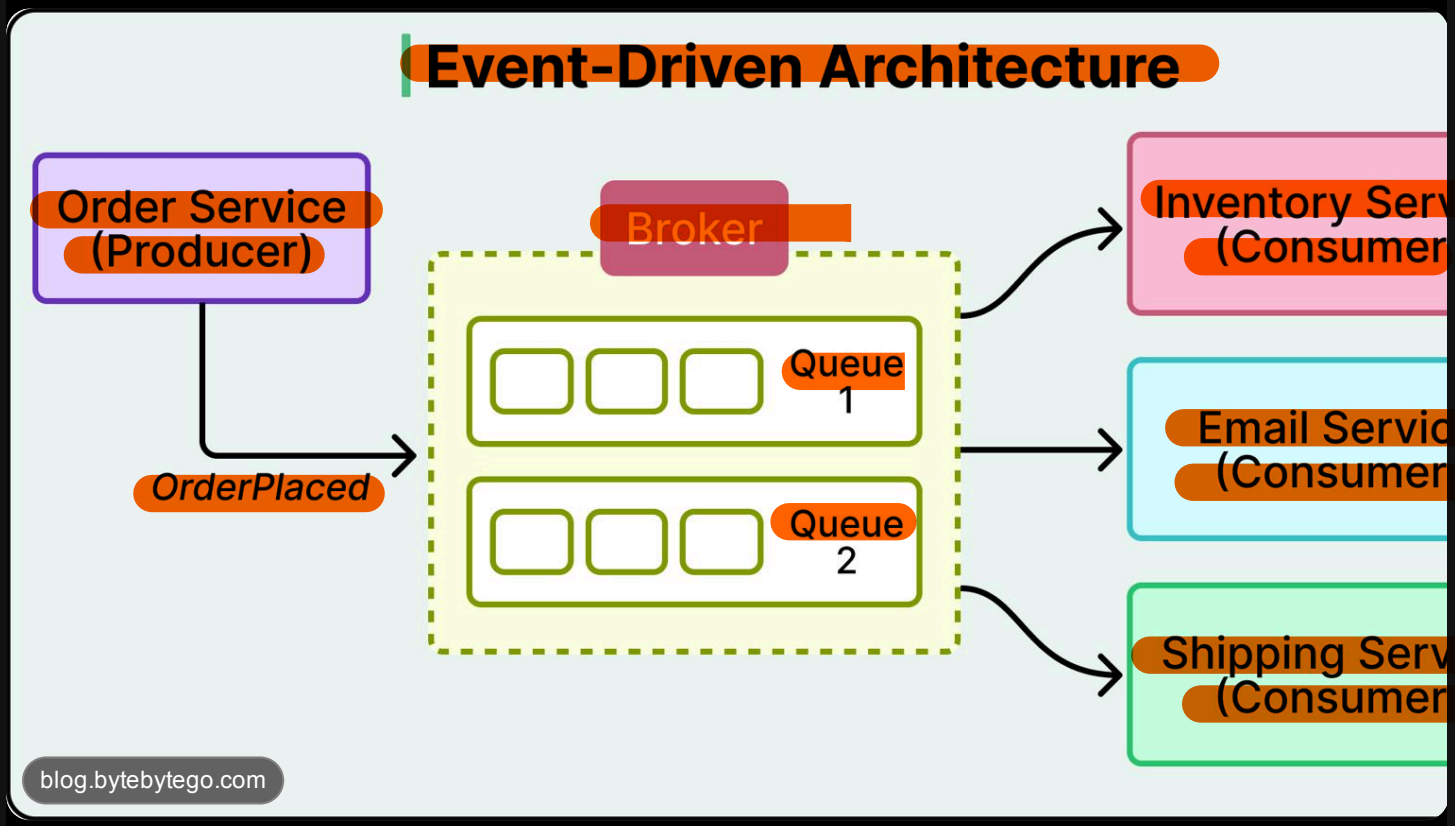
$$0.999^4 \approx 99.60\%$$

This illustrates why long chains of mandatory synchronous dependencies are dangerous.

Use synchronous calls when: the caller genuinely needs the response now.

Prefer asynchronous communication when: work can continue independently, processing is long-running, or temporary downstream outages should not immediately fail the initiating request.

3.3 Asynchronous Messaging and Event-Driven Architecture



Problem: Direct service-to-service calls create tight runtime coupling. A downstream outage can cause upstream failures.

Solution: Publish messages or domain events through a broker such as Kafka or RabbitMQ.

```
Order Service
    |
    └── OrderPlaced event
            |
        Message Broker
            |
        ┌───┴────┐
        ▼        ▼
   Inventory  Notification
    Service    Service
```

- **DSA:** Queueing theory, FIFO structures, partitioning, hashing, consumer concurrency.
- **LLD:** Producers, consumers, event schemas, acknowledgement strategies, retry handlers, idempotency.
- **HLD:** Broker topology, partition count, replication, consumer groups, retention, backpressure, replay.

Queueing theory

Let:

- λ = incoming messages per second.
- μ = processing capacity per consumer per second.
- c = number of consumers.

Approximate utilization is:

$$\rho = \frac{\lambda}{c\mu}$$

For a stable queue under ordinary assumptions, utilization must remain below 1. As utilization approaches 1, queueing delay can increase sharply.

For example, if the system receives 800 messages/s and each consumer handles 300 messages/s, three consumers have a nominal aggregate capacity of 900 messages/s. That leaves little headroom for bursts, uneven partitions, retries, or slow messages.

Trade-offs: Asynchronous messaging improves decoupling and absorbs bursts, but introduces eventual consistency, duplicate delivery, ordering constraints, and more complex debugging.

Important distinction: An event says that something happened (`OrderPlaced`). A command requests that something happen (`ReserveInventory`). The distinction helps keep event contracts meaningful.

3.4 Service Discovery and Load Balancing

Problem: Service instances are ephemeral. Containers restart, scale, and move between hosts.

Solution: Resolve logical service names to healthy instances and distribute requests.

Plain text

```
Order Service
     |
     ▼
```

```
Service Discovery / Load Balancer
    ├── Payment instance A
    ├── Payment instance B
    └── Payment instance C
```

- **DSA:** Round-robin scheduling, weighted selection, hashing, health-state lookup.
- **LLD:** Discovery client, load-balancing strategy, health checker, connection lifecycle.
- **HLD:** Kubernetes Services, DNS, service registries, ingress, proxy-based routing.

Common strategies:

Strategy	Principle	Trade-off
Round robin	Rotate through instances	Does not account for load differences
Least connections	Prefer fewer active connections	Requires accurate connection state
Weighted round robin	Allocate traffic by capacity	Weights may become stale
Consistent hashing	Map keys to stable partitions or nodes	Uneven distribution and membership changes require care

Interview distinction: Discovery tells you which instances are available; load balancing decides which instance receives a request.

4. Resilience patterns

These patterns are particularly important in Java and Spring Boot systems because network failures, slow dependencies, and thread exhaustion can cause cascading failures.

4.1 Timeout

A timeout limits how long a caller waits for a dependency.

- **DSA:** Deadline propagation, bounded waiting, cancellation, queueing delay.
- **LLD:** HTTP client timeouts, database query timeouts, interrupt/cancellation handling.
- **HLD:** End-to-end latency budgets and protection against resource exhaustion.

A timeout must account for the entire request path, not merely one HTTP connection setting.

For example, if an API has a 2-second deadline, spending 1.8 seconds on one downstream call leaves almost no time for other work.

Also distinguish connection timeout, response/read timeout, and an end-to-end deadline. They are not interchangeable.

4.2 Retry with exponential backoff and jitter

Problem: Temporary network errors or service overload cause requests to fail.

Solution: Retry selected transient failures, with bounded attempts and delays.

A common backoff formula is:

$$d_k = \min(d_{\max}, d_0 2^k)$$

With full jitter, choose a random delay:

$$D_k \sim U(0, d_k)$$

where d_0 is the initial delay, $d_{\max}$ is the cap, and k is the retry index.

- **DSA:** Exponential growth, randomized delay, attempt counters.
- **LLD:** Retry predicates, maximum attempts, deadline-aware execution, exception classification.
- **HLD:** Retry budgets, overload protection, retry ownership, fleet-wide amplification.

Example: A payment request times out. Blindly retrying could charge the customer twice if the first request succeeded but its response was lost.

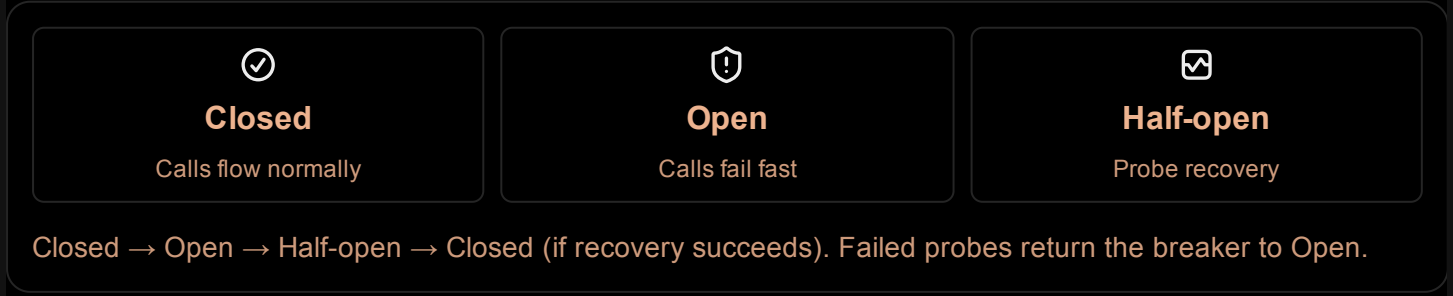
Use retries only when the operation is safe to repeat, or when the system has a robust idempotency mechanism.

Avoid retries at every layer. If three layers each allow three total attempts, a single logical operation can generate up to $3^3 = 27$ downstream attempts.

4.3 Circuit Breaker

Problem: Repeated calls to an unhealthy dependency waste threads, connections, and CPU.

Solution: Temporarily stop sending normal traffic after a failure threshold is reached.



Closed → Open → Half-open → Closed (if recovery succeeds). Failed probes return the breaker to Open.

- **DSA:** Rolling failure counters, sliding windows, state machines, time-based transitions.
- **LLD:** State enum, failure-rate calculation, permitted-probe logic, fallback and exception handling.
- **HLD:** Dependency isolation, failure thresholds, recovery traffic, metrics and alerts.

Circuit breakers are usually local to a process or client instance. They do not automatically coordinate their state across an entire fleet.

Difference from retries: Retries attempt recovery from a potentially transient failure. Circuit breakers stop calls when continued attempts are likely to be harmful.

4.4 Bulkhead

Problem: A slow dependency consumes all request-processing threads or connections, starving unrelated operations.

Solution: Isolate resources into separate pools or concurrency limits.

For example:

```
Plain text

Order Service
  ├── Payment pool:     30 concurrent calls
  ├── Inventory pool:   50 concurrent calls
  └── Shipping pool:    10 concurrent calls
```

- **DSA:** Semaphores, bounded queues, thread pools, admission control.
- **LLD:** Separate executors, semaphore guards, bounded connection pools.
- **HLD:** Resource isolation across critical dependencies and workload classes.

A semaphore limiting payment calls to 30 allows at most 30 concurrent permit holders. Excess calls must wait within a bounded policy or fail fast.

Trade-off: Isolation protects critical resources but can leave some capacity unused and introduces additional configuration.

4.5 Rate Limiting and Load Shedding

Problem: Requests exceed the capacity of an API or downstream service.

Solution: Limit admission and reject or defer excess work.

Two common algorithms:

Algorithm	DSA mechanism	Behavior
Token bucket	Token counter plus refill rate	Allows controlled bursts
Sliding window	Time buckets or timestamped requests	Limits requests over a rolling interval

For a token bucket with capacity B and refill rate r tokens/s, a request of cost q is admitted only if at least q tokens are available.

- **DSA:** Counters, timestamp windows, synchronization, atomic updates.
- **LLD:** Rate-limit filters, policy configuration, per-key state, HTTP 429 responses.
- **HLD:** Global versus per-instance quotas, distributed counters, fairness, overload behavior.

A local rate limiter on four instances may permit roughly four times the configured per-instance quota. A strict global quota needs coordinated state or a central enforcement mechanism.

Use load shedding when rejecting low-priority work is preferable to making all requests slow or unavailable.

---

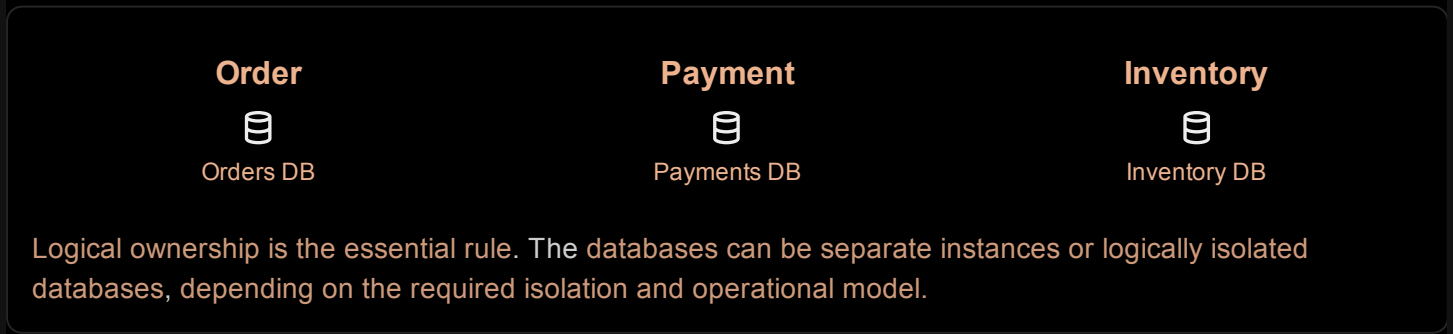
5. Data management and distributed transaction patterns

This category is one of the most important for Staff Engineer and Architect interviews. Once data is split across services, a normal database transaction cannot usually protect the entire business operation.

5.1 Database per Service

Problem: Multiple services directly access shared tables, making schema changes and deployments tightly coupled.

Solution: Each service owns its data and exposes it through APIs or events.



Logical ownership is the essential rule. The databases can be separate instances or logically isolated databases, depending on the required isolation and operational model.

- **DSA:** Index structures, partitioning, caching, query complexity, data locality.
- **LLD:** Repository interfaces, schema migrations, aggregate persistence, transaction boundaries.
- **HLD:** Ownership, independent schema evolution, cross-service queries, replication and consistency.

Trade-off: Service independence improves, but cross-service joins and atomic updates become harder.

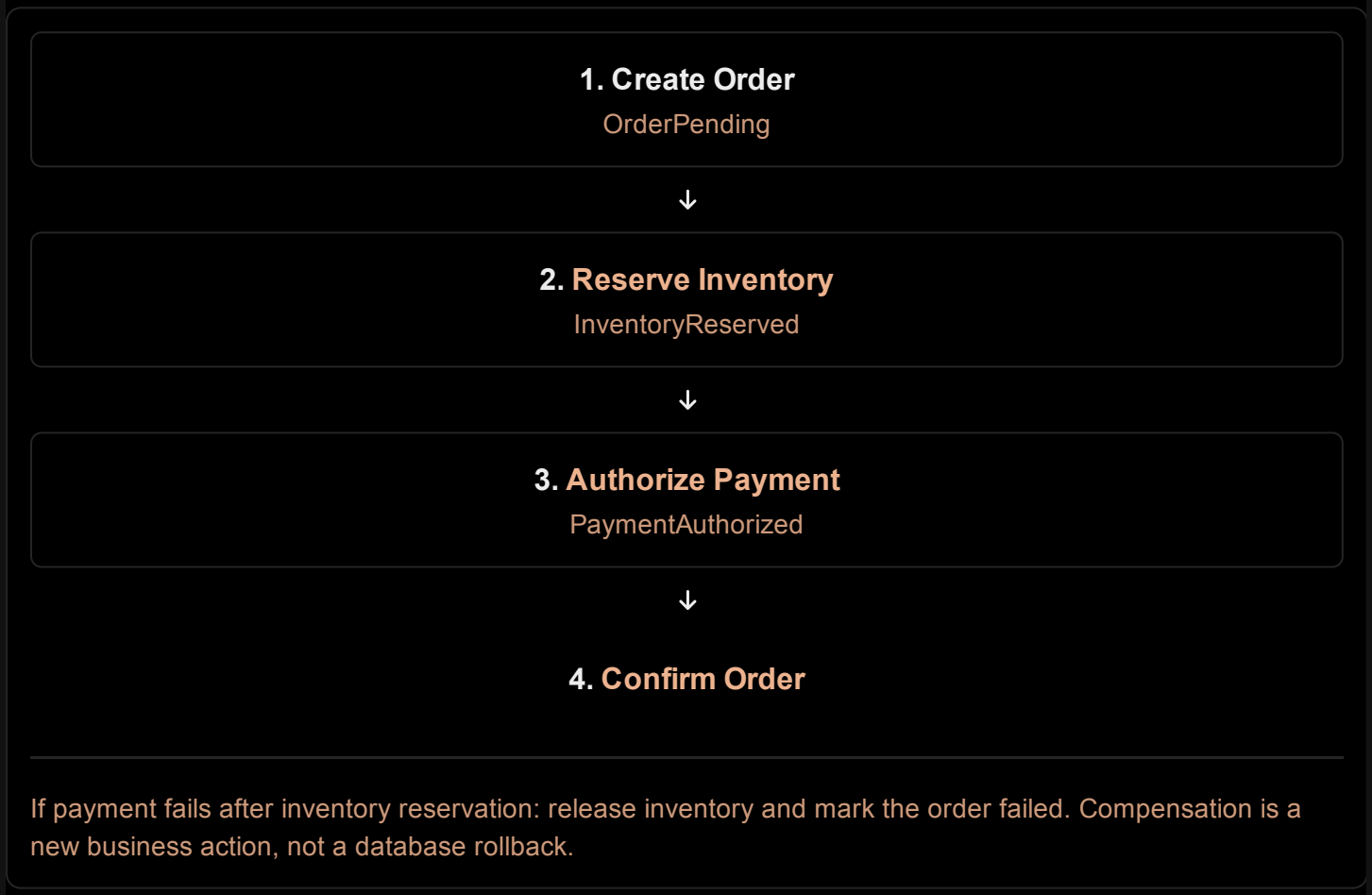
Avoid direct cross-service database writes. They bypass domain invariants and create hidden coupling.

5.2 Saga Pattern

Problem: A business transaction spans several services, each with its own database.

Solution: Split the workflow into local transactions and define compensating actions for failures.

Consider placing an order:



Two Saga styles

- **Choreography:** Services react to events and publish further events. There is no central workflow coordinator.
- **Orchestration:** A Saga orchestrator directs the workflow, tracks progress, and decides which step runs next.

DSA perspective

A Saga is naturally modeled as a finite-state machine or a directed workflow graph.

Represent the state as:

```
Plain text

PENDING
  → INVENTORY_RESERVED
  → PAYMENT_AUTHORIZED
  → CONFIRMED

INVENTORY_RESERVED
  → COMPENSATING
  → CANCELLED
```

Important mechanisms:

- State transitions with explicit guards.
- Durable workflow state.
- Idempotent command handling.
- Bounded retry policies.
- Timeouts and compensation tracking.

LLD perspective

Define a step contract:

```
Java

public interface SagaStep<C> {
    void execute(C context);
    void compensate(C context);
}
```

A production implementation also needs to model step results, durable progress, retryability, compensation failures, and idempotency. A `void` interface alone is not sufficient to represent those concerns.

HLD perspective

Decide:

- Who owns workflow state.
- How commands and events are delivered.
- What happens when a service remains unavailable.

- Whether compensation is possible.
- How operations are reconciled when external systems cannot be rolled back.

Critical insight: A Saga provides a way to manage distributed business workflows; it does not provide ACID isolation across services.

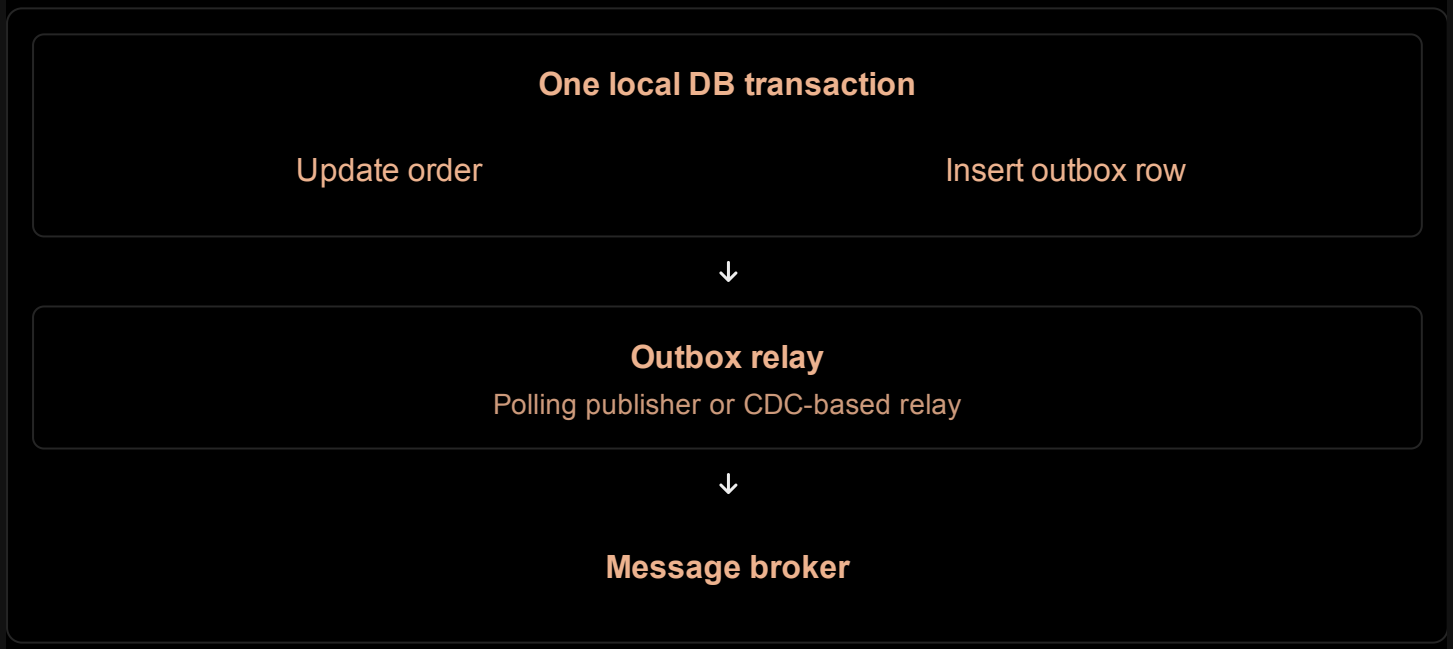
For example, a refund is not a reversal of history. It is a new financial transaction with its own status and failure modes.

5.3 Transactional Outbox

Problem: A service updates its database and publishes an event. If the database commit succeeds but event publication fails, other services never learn about the update.

This is the dual-write problem.

Solution: Save the business update and an event record in the same local database transaction. A separate publisher relays the event to the broker.



- **DSA:** Work queues, batching, pagination, deduplication, ordered processing.
- **LLD:** Outbox table, transaction boundaries, relay component, retry handling, event identifiers.
- **HLD:** CDC versus polling, replication lag, broker durability, retention, monitoring and recovery.

Example table:

```sql
CREATE TABLE outbox (
    event_id UUID PRIMARY KEY,
    aggregate_id UUID NOT NULL,
```

```
    event_type VARCHAR(100) NOT NULL,
    payload JSONB NOT NULL,
    created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
    published_at TIMESTAMPTZ
);
```

The event record must be inserted in the same transaction as the business change.

Important: The Outbox pattern does not guarantee exactly-once delivery to consumers. A relay can publish an event and crash before marking it published, causing a duplicate. Consumers still need idempotency.

5.4 Idempotent Consumer and Inbox

Problem: A broker redelivers a message, or a consumer processes a message but crashes before acknowledging it.

Solution: Give each event a stable identifier and persist its processing status transactionally.

Plain text

```
Receive event
    ↓
Check processed-event record
    ↓
Already processed? ── Yes → Return safely
    |
    No
    ↓
Update business state + insert processed-event record
in the same database transaction
    ↓
Commit
    ↓
Acknowledge message
```

- **DSA:** Hash-based deduplication, set membership, retention and eviction policies.
- **LLD:** Inbox/processed-message table, transaction management, unique constraints.
- **HLD:** Redelivery, consumer restarts, replay, deduplication scope and retention.

Example:

SQL

```
CREATE TABLE processed_events (
    consumer_name VARCHAR(100) NOT NULL,
    event_id UUID NOT NULL,
    processed_at TIMESTAMPTZ NOT NULL DEFAULT now(),
    PRIMARY KEY (consumer_name, event_id)
);
```

Use the unique constraint to arbitrate concurrent attempts. Do not rely only on an in-memory set because it disappears on restart and is not shared across instances.

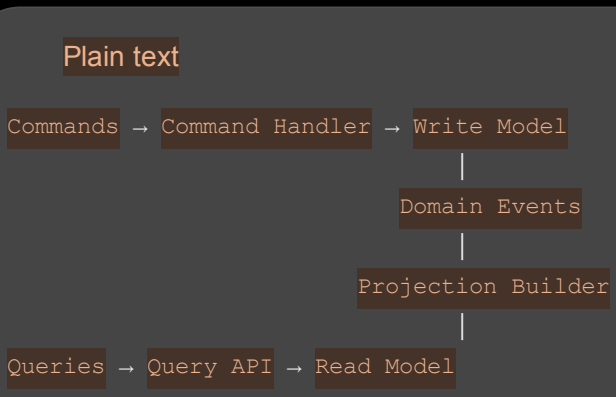
Exactly-once nuance: A broker may provide exactly-once guarantees within a defined processing boundary. That does not automatically make an external payment, email, or database side effect exactly once. End-to-end guarantees require explicit coordination and idempotency at the relevant boundary.

5.5 CQRS — Command Query Responsibility Segregation

Problem: The same data model may not efficiently support both transactional writes and complex, high-volume reads.

Solution: Separate command handling from query handling, potentially using different models and data stores.

```
Plain text
```



- **DSA:** Indexes, denormalization, aggregation, incremental projection updates.
- **LLD:** Command handlers, query handlers, DTOs, projection consumers, validation.
- **HLD:** Read replicas, materialized views, projection lag, rebuilding and reconciliation.

For example, Order Service may maintain normalized order records while a customer order-history view stores denormalized data for fast reads.

Trade-off: CQRS can improve read performance and independent scaling, but adds synchronization and operational complexity.

CQRS does not require separate databases, and it does not necessarily require Event Sourcing.

5.6 Event Sourcing

Problem: A conventional database often stores the latest state but not the full sequence of domain changes as the primary source of truth.

Solution: Persist an append-only sequence of events and reconstruct state by replaying them.

```
Plain text

OrderCreated
    → ItemAdded
    → PaymentAuthorized
    → OrderConfirmed
```

The current order state is derived from these events.

- **DSA:** Event folding, replay, snapshots, stream partitioning.
- **LLD:** Event schema versioning, aggregate reconstruction, command validation, snapshots.
- **HLD:** Event store durability, retention, replay, projections, schema evolution.

A simple state reconstruction is:

$$S_n = \text{fold}(S_0, e_1, e_2, \ldots, e_n)$$

Here, each event transforms the previous state into the next state.

Trade-off: Event Sourcing provides an auditable history and supports new projections, but makes schema evolution, replay, debugging, and corrections more complicated.

Use it when historical state transitions are fundamental to the domain—not merely because events are already used for messaging.

---

6. Other high-value data patterns

Pattern	DSA perspective	LLD perspective	HLD perspective
Database per Service	Indexes, partitioning	Repositories, schema ownership	Data isolation and independent evolution

Saga	State machines, workflow graphs	Step execution and compensation	Distributed transaction management
Outbox	Queueing, batching	Local transaction and relay	Reliable event publication
Inbox / Idempotency	Hash sets, unique keys	Deduplication transaction	Duplicate delivery and replay
CQRS	Query optimization, projections	Command/query handlers	Independent read/write scaling
Event Sourcing	Folding, snapshots	Event persistence and reconstruction	Replay, auditing, schema evolution
Materialized View	Incremental aggregation	Projection updater	Fast reads with stale-data risk
Cache-Aside	Hash maps, eviction policies	Cache abstraction and TTL	Latency, invalidation, consistency

Cache-Aside in microservices

The service checks the cache first, loads from the database on a miss, and populates the cache.

- **DSA:** Hash lookup, LRU/LFU eviction, TTL, cache stampede control.
- **LLD:** Cache wrapper, serialization, expiration policy, failure handling.
- **HLD:** Cache distribution, invalidation, hot keys, stale reads, capacity.

The hardest problem is usually invalidation and consistency—not implementing a cache lookup.

---

7. Security, observability, and deployment patterns

These are cross-cutting concerns. A microservices architecture is incomplete if it focuses only on service communication and data storage.

7.1 Service-to-Service Security

Problem: Services need to authenticate callers, authorize operations, and protect traffic without relying on network location alone.

Patterns and mechanisms:

- OAuth 2.0 access tokens and appropriate authorization policies.
- Mutual TLS (mTLS) for service identity and transport protection.
- Least-privilege service identities.
- Zero Trust: authenticate and authorize each relevant communication boundary.

Perspective	What matters

DSA	Token validation, cache lookups, cryptographic verification cost
LLD	Authentication filters, authorization policies, token propagation, credential rotation
HLD	Identity provider, trust boundaries, key management, service mesh integration

In Spring Boot, Spring Security can enforce authentication and authorization. A service mesh can manage mTLS between workloads, but it does not replace application-level authorization.

7.2 Distributed Tracing and Correlation IDs

Problem: One logical request may cross ten services and several asynchronous queues. Individual logs cannot explain the entire request lifecycle.

Solution: Propagate trace context across network calls and message headers.

```
Plain text

Incoming Request
  └── Order Service          [Trace ID: T1]
      ├── Inventory          [Trace ID: T1]
      ├── Payment            [Trace ID: T1]
      └── Publish Event      [Trace ID: T1]
          └── Notification Consumer [Trace ID: T1]
```

- **DSA:** Trace-context propagation, bounded buffering, sampling decisions.
- **LLD:** Instrumentation, context propagation, structured logs, span creation.
- **HLD:** OpenTelemetry, trace collectors, storage, sampling, service-level objectives.

Use a trace ID for the distributed operation and span IDs for individual operations. Correlation IDs are also useful for linking business operations, but are not necessarily interchangeable with tracing identifiers.

In Java/Spring Boot, OpenTelemetry instrumentation can connect HTTP requests, database calls, and message processing.

7.3 Sidecar and Ambassador

These patterns move selected infrastructure functionality out of application business code.

- **Sidecar:** A helper process runs alongside a service instance, often for proxying, telemetry, or security.
- **Ambassador:** A local proxy mediates outbound communication to external systems.

DSA: Connection pools, routing tables, buffering, and protocol processing.

LLD: Proxy configuration, local communication interfaces, health checks, and lifecycle management.

HLD: Deployment topology, service mesh, resource overhead, network policy, and operational ownership.

Trade-off: Infrastructure concerns become more consistent, but extra processes introduce latency, resource consumption, and additional failure modes.

7.4 Health Checks, Canary, and Blue-Green Deployment

- **Liveness:** Is the process functioning, or should it be restarted?
- **Readiness:** Should the instance receive traffic?
- **Startup probe:** Has the application finished starting?

A readiness failure should generally remove an instance from normal traffic, not automatically restart it.

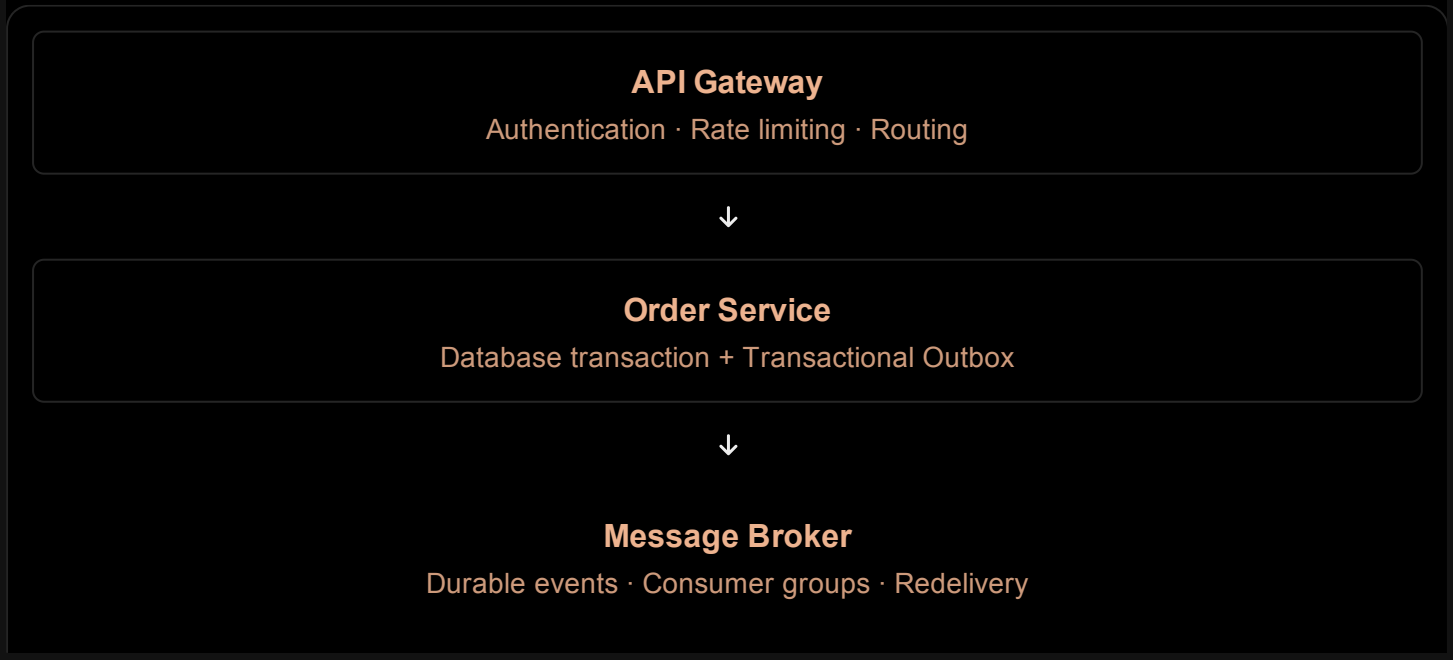
Canary deployment gradually exposes a new version to a small portion of traffic. Blue-green deployment maintains two environments and switches traffic between them.

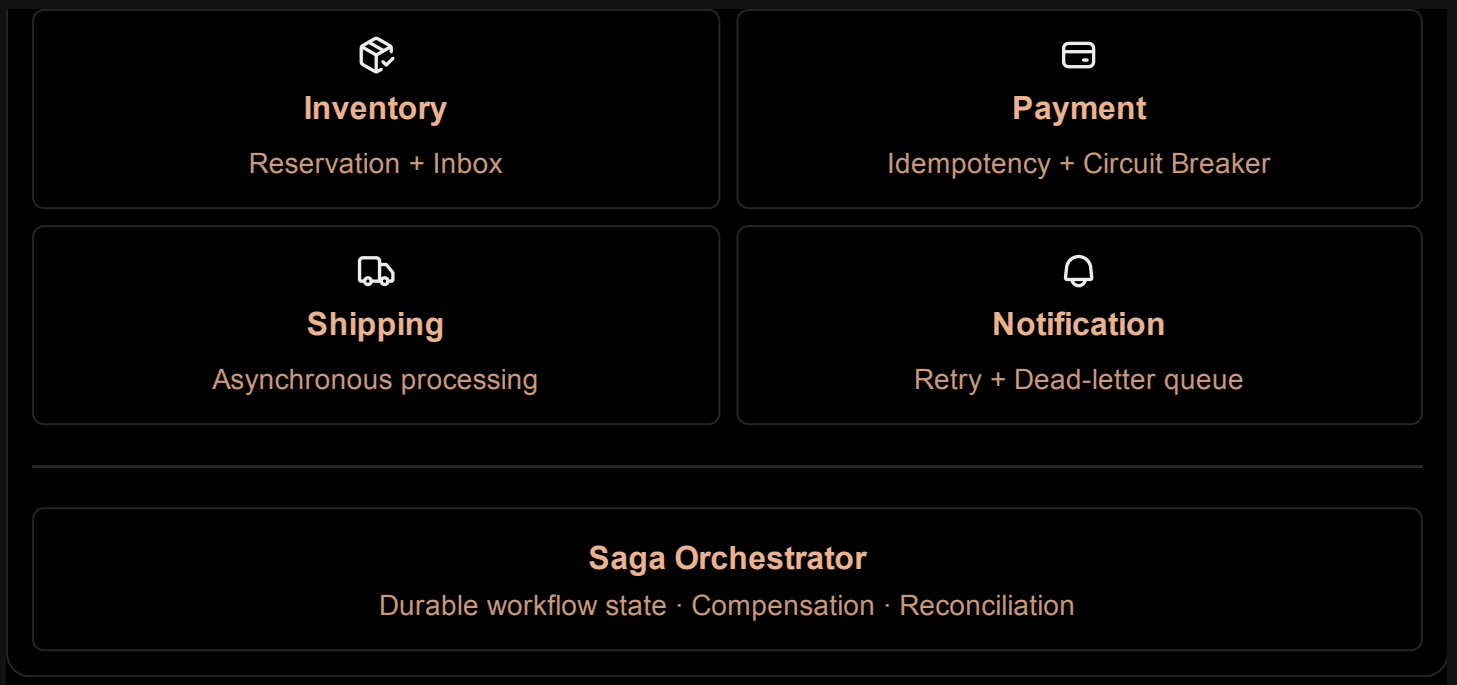
- **DSA:** Traffic percentages, routing rules, health-state tracking.
- **LLD:** Version-compatible APIs, readiness endpoints, graceful shutdown.
- **HLD:** Rollback, database compatibility, progressive delivery, error-budget monitoring.

A deployment rollback does not automatically roll back database changes. Prefer backward-compatible schema migrations and expand-contract migration strategies.

8. How the patterns work together in a real system

Consider a high-volume order processing platform.





Supporting every service are service discovery, workload identity, health checks, metrics, logs, and distributed tracing.

Walk through a failure

Suppose the Order Service commits an order, but the broker is temporarily unavailable.

1. The order and its outbox record commit together.
2. The publisher retries delivery when the broker recovers.
3. The broker may deliver the event more than once.
4. The consumer's inbox and business update transaction prevent duplicate processing.
5. The Saga tracks the business workflow and handles timeouts or failed steps.
6. Metrics and tracing expose publication lag, retries, and workflow failures.

This is the key architectural principle: **no single pattern solves distributed reliability. Reliability emerges from correctly composing patterns and explicitly defining their guarantees.**

---

9. DSA problems hidden inside microservices patterns

For your MAANG+ senior backend and architect preparation, practice these as algorithmic exercises in addition to reading architecture descriptions.

Problem	DSA concepts	Relevant patterns
Implement a token bucket	Counters, time, concurrency	Rate Limiter
Implement a sliding-window limiter	Queue, deque, timestamps	Rate Limiter
Build a circuit breaker	State machine, rolling statistics	Circuit Breaker

Build a bounded worker pool	Blocking queues, semaphores	Bulkhead
Deduplicate millions of events	Hashing, storage, retention	Inbox / Idempotency
Partition events by aggregate ID	Hashing, partition assignment	Event Streaming
Reconstruct entity state from events	Folding, ordering	Event Sourcing
Execute a dependency workflow	Directed graph, topological ordering	Saga / Workflow
Find cyclic service dependencies	DFS, SCC algorithms	Service Decomposition
Schedule retries	Priority queue, backoff	Retry / Delayed Messaging
Build a cache with eviction	Hash map + doubly linked list	Cache-Aside
Process a bounded backlog	Queueing, admission control	Backpressure

Three example implementations to master

A. LRU cache

Use a hash map and doubly linked list.

- `get(key)`: $O(1)$ average.
- `put(key, value)`: $O(1)$ average.
- Capacity is bounded by eviction of the least recently used entry.

Microservices relevance: local caching, metadata caches, and rate-limit state.

B. Circuit breaker

Maintain states `CLOSED`, `OPEN`, and `HALF_OPEN`, together with failure statistics and a recovery deadline.

- Evaluate failures using a rolling window or count-based threshold.
- Reject requests while open.
- Permit a limited number of probes while half-open.
- Reopen on failed probes.

The difficult parts are concurrent state transitions, stale statistics, and controlling probe concurrency.

C. Saga workflow engine

Represent each step with its forward action, compensation action, retry policy, and durable status.

A workflow can be represented as a directed graph if branching and parallel steps are required. A simple sequential Saga can use a list of steps and a state machine.

The important edge cases are process crashes, duplicate events, partial compensation, timeouts, and recovery after restart.

---

10. Interview decision matrix

Use the problem statement to identify the pattern rather than choosing a pattern by name.

If the problem says…	Consider…	Critical question
“Services depend on each other”	API Gateway, service discovery	Where is routing and discovery performed?
“The downstream service is slow”	Timeout, Bulkhead	Can it exhaust all available resources?
“Temporary failures occur”	Retry, Circuit Breaker	Is retry safe, and when should it stop?
“One business operation spans databases”	Saga	What are the compensation rules?
“A database update and event must stay consistent”	Transactional Outbox	What happens if publishing fails after commit?
“Messages may be delivered more than once”	Idempotent Consumer, Inbox	Is deduplication durable and transactional?
“Reads dominate writes”	Caching, CQRS	What staleness is acceptable?
“The complete change history matters”	Event Sourcing	Is the event log the source of truth?
“A new service must replace a legacy module safely”	Strangler Fig	How will data ownership and rollback work?
“One tenant can overwhelm the system”	Rate Limiting, Bulkhead	Are limits local, per-tenant, or global?
“A request crosses multiple services”	Distributed Tracing	Can you correlate all operations and failures?

11. Recommended learning order

For mastery, study patterns in the following sequence rather than learning them as an isolated glossary.

1. **Service boundaries:** DDD, bounded contexts, Database per Service, modular monolith, Strangler Fig.
2. **Communication:** REST, gRPC, API Gateway, service discovery, event-driven architecture, Kafka and RabbitMQ fundamentals.
3. **Resilience:** Timeouts, retries with jitter, Circuit Breaker, Bulkhead, Rate Limiting, backpressure.

4. **Distributed consistency:** Saga, Outbox, Inbox, idempotency, ordering, dead-letter queues, reconciliation.
5. **Data architecture:** CQRS, Event Sourcing, materialized views, caching, read replicas, partitioning.
6. **Production architecture:** OAuth 2.0, mTLS, OpenTelemetry, health checks, Kubernetes deployment strategies, SLOs.
7. **System design practice:** Design an order-processing system, then a payment platform, then a high-throughput event-processing system.

For each pattern, work through four questions:

1. What failure or design constraint does it solve?
2. What is the core algorithm or state machine?
3. How would you implement it in Java/Spring Boot and test its failure modes?
4. What are its system-wide trade-offs and operational requirements?

12. Resources

- Microservices Patterns by Chris Richardson — Saga, Outbox, API Gateway, and distributed data management.
- Building Microservices by Sam Newman — service boundaries, communication, deployment, and operational trade-offs.
- Designing Data-Intensive Applications by Martin Kleppmann — replication, transactions, logs, streams, and consistency.
- Domain-Driven Design Distilled by Vaughn Vernon — bounded contexts and domain modeling.
- Spring Cloud official documentation — gateway, discovery, resilience, and distributed configuration.
- OpenTelemetry official documentation — traces, metrics, logs, and context propagation.

My recommendation is to use *Microservices Patterns* as the primary pattern reference and *Designing Data-Intensive Applications* to develop the deeper distributed-systems reasoning needed for Staff-level interviews.

The goal is to progress from knowing what a pattern is, to implementing it correctly, to defending why it is appropriate for a particular system.